Software Engineering

Chapter 5: Software Architecture & Design

Prof. Nien-Lin Hsueh
Department of Information Engineering and Computer Science
Feng Chia University

Chapter 5: Roadmap & Core Curriculum

Part 1: Architectural Design

- **5.1 Foundations of Architecture:** The critical bridge from requirements to design, abstraction in the small vs. large.
- **5.2 Kruchten's 4+1 Views:** Logical, process, development, physical views unified by central scenarios.
- **5.3 Classic Architectural Patterns:** Layered, repository, client-server, pipe-and-filter, MVC, and microservices.
- **5.4 Architecture & Quality Attributes:** Non-functional trade-offs: performance, security, availability, and safety.

Part 2: Detailed Design & Principles

- **5.5 Fundamentals of Software Design:** Design activities, modularity, information hiding, high cohesion & low coupling.
- **5.6 SOLID Design Principles:** SRP, OCP, LSP, ISP, and DIP with real-world refactoring examples.
- **5.7 AI in Architecture & Design:** AI copilots for trade-off analysis, refactoring, code smell detection, and risks.
- **5.8 Conceptual Recap & Synthesis:** Core principles review, fill-in-the-blank self-assessment quiz, and literature.

Focus Questions

- What is **Software Architecture** and why does it serve as the critical link between requirements and detailed design?
- How does **Architecture in the Small** differ from **Architecture in the Large**?
- What are Kruchten's **4+1 Architectural Views** and how do they satisfy diverse stakeholder needs?
- How do classic **Architectural Patterns** (Layered, Repository, Client-Server, Pipe & Filter, MVC, Microservices) structure software?
- Why is **High Cohesion and Low Coupling** the universal golden rule of software engineering?
- How do the **5 SOLID Principles** (SRP, OCP, LSP, ISP, DIP) prevent architectural rot and rigidity?
- How can **Generative AI** assist software architects while safeguarding against architectural blind spots?

5.1 Foundations of Software Architecture

> *"Software architecture is the set of design decisions which, if made incorrectly, may cause your project to fail."*
> *— Eoin Woods*

What is Software Architecture?

> *The fundamental organization of a software system embodied in its components, their relationships to each other and the environment, and the principles guiding its design and evolution.*
>
> ”

- **The Critical Bridge Role:**
  - Connects **Requirements Engineering** (*what the customer needs*) to **Detailed Software Design** (*how components are constructed*).
  - Identifies principal structural components and defines their public communication interfaces.
- **Architecture as Design Decisions:**
  - Architecture represents the early, high-impact design decisions that are **costliest to change** once implementation begins.

[SYSTEM_CONTEXT] // DEFINING THE CHASM

Connecting the ‘What’ to the ‘How’





Software Design

Requirements Engineering (The What)

// Documenting customer needs & specifications

Architecture: Identifying principal structural components and their relationships.

// Defining high-level structure, modules, interfaces & interactions. Ensuring scalability, security, and performance.

Component Design: Structuring internal algorithms, classes, and logic.

// Detailing low-level logic, data structures, functions, and specific algorithmic implementations.

Implementation (The Code)

// Generating executable software & algorithms

NotebookLM

Architectural Abstraction Levels

- **Architecture in the Small:**
  - Concerned with the design of individual programs, microservices, or client applications.
  - Focuses on how a single program is decomposed into sub-components, packages, and object classes.
  - Directly influences internal code readability, testability, and refactoring velocity.
- **Architecture in the Large:**
  - Concerned with complex enterprise systems composed of interacting multi-system networks.
  - Systems are distributed across multiple servers, cloud regions, and external vendor platforms.
  - Encompasses legacy database integration, message queues, and cross-organizational API contracts.

Advantages of Explicit Software Architecture

1. **Stakeholder Communication:**
   - High-level architectural diagrams act as a common focus of discussion between technical teams, business executives, and external clients without getting bogged down in code syntax.
2. **System Analysis & Non-Functional Verification:**
   - Enables early verification of whether the proposed design can satisfy critical **non-functional requirements** (e.g., peak latency, fault tolerance, compliance).
3. **Large-Scale Reuse:**
   - Architectural patterns and component frameworks can be systematically reused across entire product families in the same domain.

5.2 Kruchten's 4+1 Architectural Views

> *"If you look at an architecture from only one perspective, you are blind to three-quarters of its reality."*
> *— Philippe Kruchten*

Kruchten's 4+1 Architectural View Model

- **1. Logical View (Domain / Object Perspective):**
  - Shows key domain abstractions, object classes, and relationships (*Target Audience: System Analysts & Customers*).
- **2. Process View (Concurrency / Runtime Perspective):**
  - Shows runtime processes, threads, concurrency, and inter-process communication (*Target Audience: Integrators & Performance Engineers*).
- **3. Development View (Module / Implementation Perspective):**
  - Shows software packages, build artifacts, libraries, and source code hierarchies (*Target Audience: Software Developers*).
- **4. Physical View (Deployment / Infrastructure Perspective):**
  - Shows physical server hardware, cloud containers, and network topology (*Target Audience: DevOps & System Administrators*).
- **+1 Scenarios (Use Cases):**
  - Unifies and validates all four views by demonstrating how they execute real user journeys together.

[MACRO_LEVEL] // THE 4+1 VIEW MODEL

Five Perspectives, One Unified Blueprint

Logical View:
Object classes and key domain abstractions.

Process View:
Runtime interacting processes and threads.



Development View:
Code structure, packages, and sub-systems.

Physical View:
Hardware deployment and network distribution.

[SYS_NOTE: Solves Concept Check 1 - Maps components to runtime hardware]

NotebookLM

Concept Check Question 1

Which architectural view in Kruchten's 4+1 View Model illustrates how software components are deployed across physical hardware servers, cloud containers, and network topology at runtime?



- **A.** Logical View
- **B.** Process View
- **C.** Development View
- **D.** Physical (Deployment) View

5.3 Classic Architectural Patterns

> *"Good architects borrow; great architects reuse proven architectural patterns."* „

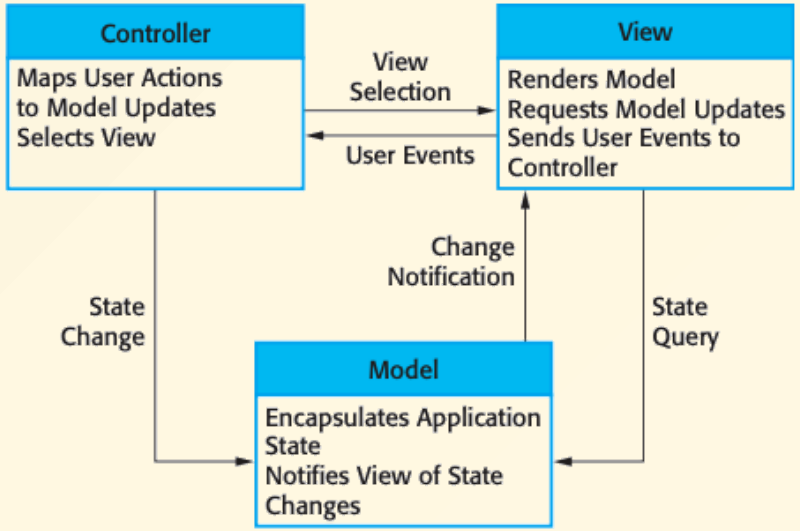
What is an Architectural Pattern?

> *A stylized, tested description of good design practice that has been tried and tested in different environments and is applicable to common structural scenarios.*

- **Pattern Anatomy:**
  - **Pattern Name:** Meaningful identifier (*MVC*, *Layered*, *Repository*).
  - **Context & Problem:** Common structural challenge the pattern addresses.
  - **Solution:** Description of principal components and their communication relationships.
  - **Trade-offs:** Clear articulation of advantages and architectural penalties.
- **Why Patterns Matter:**
  - Establishes a common vocabulary across engineering teams.
  - Prevents teams from reinventing flawed architectural solutions.

1. Model-View-Controller (MVC) Pattern

- **Component Separation:**
  - **Model:** Manages core business domain logic, data state, and persistence rules.
  - **View:** Renders user interfaces and presents data to human users.
  - **Controller:** Intercepts user inputs (clicks, HTTP requests), invokes Model updates, and selects Views.
- **Core Advantage:**
  - **Decouples Data from Presentation:** Multiple simultaneous views (web dashboard, mobile JSON API) can observe the same underlying model without modifying business logic.



2. Layered Architecture Pattern

- **Stacked Abstraction Layers:**
  - Organizes system functionality into ordered layers. Each layer provides services to the layer directly above it, and relies only on the layer directly below it.
- **Standard Enterprise Layers:**
  - **Presentation Layer:** UI, HTTP endpoints, view controllers.
  - **Application / Service Layer:** Workflow orchestration, transaction boundaries.
  - **Domain / Business Layer:** Pure domain logic, business rules, entities.
  - **Infrastructure / Data Layer:** Database access, external third-party APIs.
- **Key Advantage:**
  - **Modifiability & Isolation:** An entire layer can be replaced (e.g., swapping Oracle for PostgreSQL) without breaking upper business layers, provided interface contracts remain stable.

3. Repository Architecture Pattern

- **Centralized Data Sharing:**
  - Sub-systems exchange large volumes of complex data through a shared, centralized repository or data store.
  - Sub-systems operate independently; interactions occur strictly through repository state modifications.
- **Prominent Examples:**
  - Modern IDEs (VS Code language servers, Eclipse workspace repositories).
  - Computer-Aided Design (CAD) engineering suites.
  - Electronic Medical Health Record (EHR) central patient databases.
- **Architectural Trade-offs:**
  - *Pro:* Efficient for sharing massive, interconnected data sets.
  - *Con:* The central repository is a potential **Single Point of Failure (SPOF)** and performance bottleneck.

4. Client-Server & Pipe-and-Filter Patterns

- **Client-Server Architecture:**
  - Functionality is distributed between **Servers** offering specialized services (database, compute, auth) and **Clients** requesting services over a network.
  - *Trade-off:* High scalability across commodity hardware, but vulnerable to network latency and server connection saturation.
- **Pipe-and-Filter Architecture:**
  - Processing is organized as a discrete sequence of transformations (**Filters**) connected by data streams (**Pipes**).
  - Output of filter N becomes the input stream of filter $N+1$.
  - *Classic Example:* Unix terminal commands (`cat log.txt | grep 'ERROR' | sort | uniq -c`).
  - *Best Suited For:* Batch billing processing, data analytics pipelines, compiler toolchains.

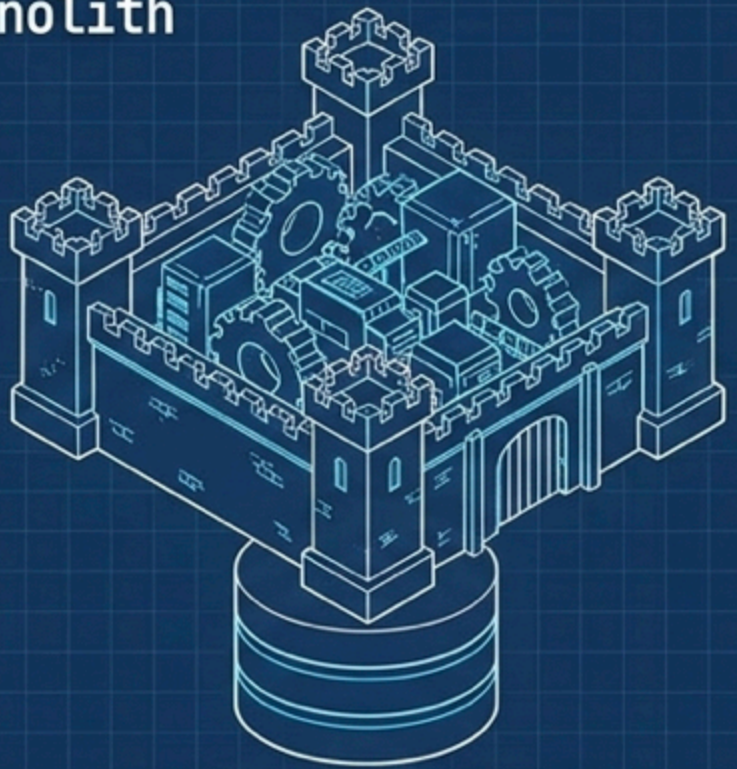
5. Microservices Architecture (MSA) vs. Monolith

- **Monolithic Architecture:**
  - All features, business logic, and database schemas packaged and deployed as a single unified executable binary.
  - *Pro:* Simple to develop and deploy early; zero network serialization overhead.
  - *Con:* Scaling requires duplicating the entire application; single code bug can crash the entire system.
- **Microservices Architecture:**
  - Decomposes the application into a suite of small, independently deployable services organized around **business capabilities**.
  - Each microservice owns its **isolated private database** (decentralized data management).
  - Services communicate asynchronously (Kafka/RabbitMQ) or via lightweight REST/gRPC APIs.

[MACRO_LEVEL] // THE SCALE SPLIT

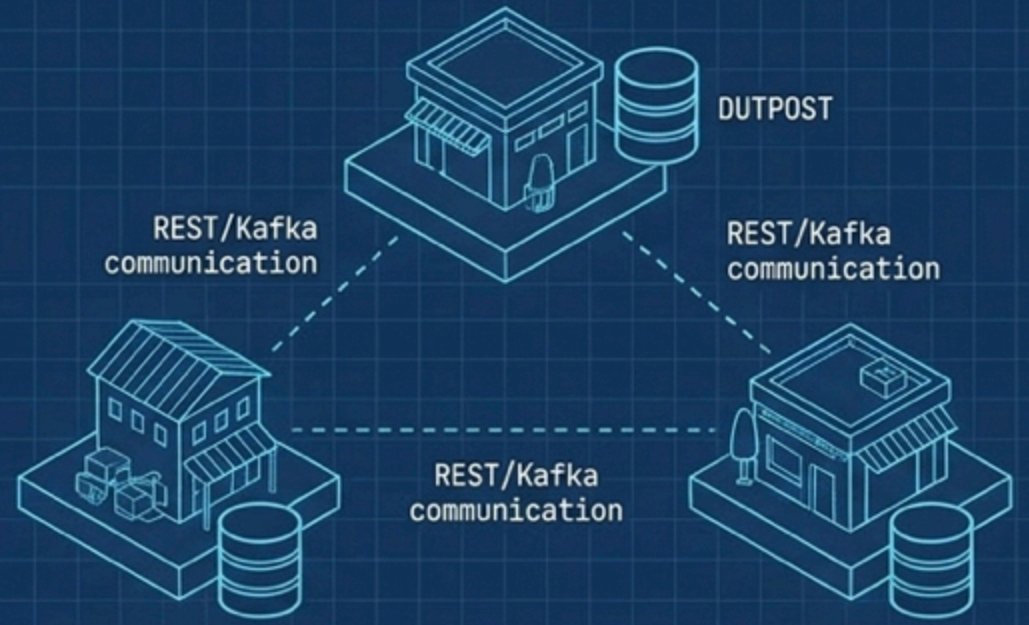
Monolithic Fortresses vs. Microservice Networks

The Monolith



SINGULAR DB VAULT

Microservices Architecture (MSA)



Simplicity

Resilience & Independent Scaling

Diagnostic Bottom Bar

NotebookLM

Concept Check Question 2

Which classic architectural pattern explicitly separates user interface presentation and layout from underlying business domain data management and user input event processing?



- **A.** Pipe and Filter Pattern
- **B.** Model-View-Controller (MVC) Pattern
- **C.** Repository Pattern
- **D.** Client-Server Pattern

5.4 Architecture & System Quality Attributes

> *"Architecture is where requirements meet reality through trade-off analysis."*

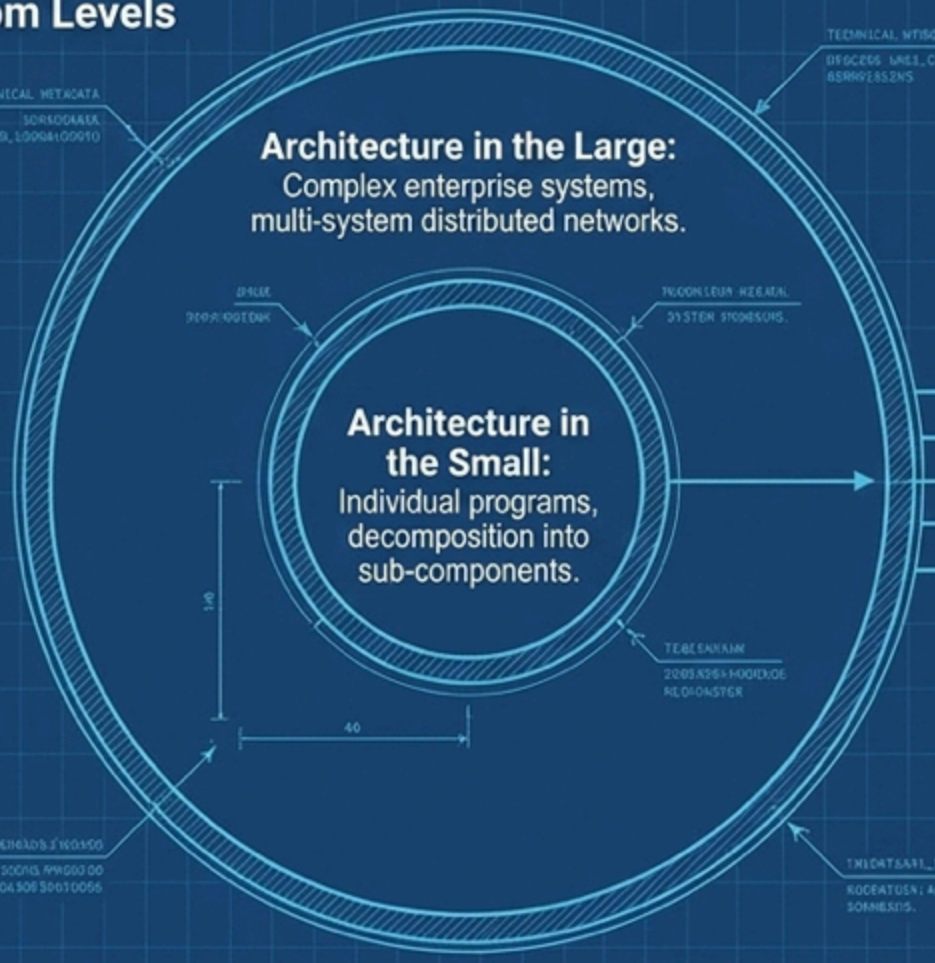
Architecture Drives System Quality Attributes

- **Performance:**
  - *Architectural Strategy:* Localize critical operations within coarse-grained components; minimize cross-network calls; deploy distributed caching layers (Redis).
- **Security:**
  - *Architectural Strategy:* Employ layered defense-in-depth; enclose high-value data assets in inner protected layers; enforce perimeter API gateways.
- **Safety:**
  - *Architectural Strategy:* Isolate safety-critical logic into independent, physically redundant sub-systems to prevent error propagation.
- **Availability:**
  - *Architectural Strategy:* Eliminate single points of failure (SPOF); implement redundant server instances, automated health checks, and active-active failover.
- **Maintainability:**
  - *Architectural Strategy:* Decompose into fine-grained, loosely coupled modules communicating strictly through stable abstract interfaces.

[MACRO_LEVEL] // SYSTEM CHARACTERISTICS

Deliberate Structure Drives System Behavior

Zoom Levels



Performance
Driven by localized operations and minimized communication.

Security
Driven by layered architectures protecting inner data assets.

Safety
Driven by isolating critical features in dedicated sub-systems.

Availability
Driven by redundant components and automated failover.

Maintainability
Driven by fine-grained, loosely coupled components.

NotebookLM

5.5 Fundamentals of Software Design

> *"There are two ways of constructing a software design: One way is to make it so simple that there are obviously no deficiencies, and the other way is to make it so complicated that there are no obvious deficiencies."*
> *— C. A. R. Hoare*

4 Fundamental Software Design Activities

1. **Architectural Design (Macro Structure):**
   - Identifying principal sub-systems, service boundaries, and communication pipelines.
2. **Interface Design (Contract Specification):**
   - Defining precise, unambiguous public contracts between modules (REST APIs, interface methods).
3. **Component / Detailed Design (Micro Structure):**
   - Designing internal class structures, algorithms, data structures, and private helper logic.
4. **Database & Data Structure Design (Persistence):**
   - Designing relational entity-relationship schemas, document collections, or object graphs.

Core Design Tenets: Modularity, Abstraction & SoC

- **1. Abstraction (抽象化):**
  - Hiding low-level implementation mechanics behind clean, high-level conceptual interfaces. Allows developers to reason about systems without mental overload.
- **2. Encapsulation & Information Hiding (封裝與資訊隱藏):**
  - *David Parnas's Principle:* Bundling state and behavior inside a module while concealing private variables. Prevents external code from introducing illegal state side-effects.
- **3. Separation of Concerns (SoC / 關注點分離):**
  - Decomposing software into distinct features that overlap as little as possible (e.g., decoupling UI rendering logic from SQL database queries).
- **4. Modularity (模組化):**
  - Partitioning systems into self-contained, independently testable, and replaceable units.

The Golden Rule: High Cohesion & Low Coupling

- **Cohesion (凝聚度):**
  - Measures how strongly related and focused the internal responsibilities of a single class or module are.
  - **Engineering Goal: High Cohesion** (A class should do one thing, and do it completely).
- **Coupling (耦合度):**
  - Measures the degree of direct inter-dependence between separate modules.
  - **Engineering Goal: Low Coupling** (Modules interact via abstract interfaces with minimal knowledge of internal mechanics).
- **The Universal Golden Rule:**

> ***Maximize Cohesion, Minimize Coupling!***
> *High cohesion makes code easy to understand; low coupling prevents changes in one module from rippling across the entire codebase.*

Cohesion & Coupling: Bad vs. Good Design

```
// ❌ BAD: Low Cohesion, High Coupling ("God Class")
class OrderManager {
    public void processOrder() {
        // Direct MySQL SQL query string execution
        // Business discount arithmetic logic
        // Raw HTML string formatting for invoice
        // Low-level SMTP socket connection to send email
    }
}
```

```
// ✅ GOOD: High Cohesion, Low Coupling (Decoupled Single-Purpose Classes)
class OrderProcessor {
    private final DiscountCalculator discountCalculator;
    private final OrderRepository orderRepository;
    private final NotificationService notificationService;

    public OrderProcessor(DiscountCalculator dc, OrderRepository repo, NotificationService ns) {
        this.discountCalculator = dc;
        this.orderRepository = repo;
        this.notificationService = ns;
```

Concept Check Question 3

In software design, what is the primary operational benefit achieved by combining **High Cohesion** with **Low Coupling**?

- **A.** It maximizes CPU execution speed by inlining all functions into a single binary.
- **B.** It creates modular code where modifications to one class have minimal ripple effects on others.
- **C.** It mandates that all application data structures be stored in global memory spaces.
- **D.** It eliminates the need for automated integration testing in production pipelines.

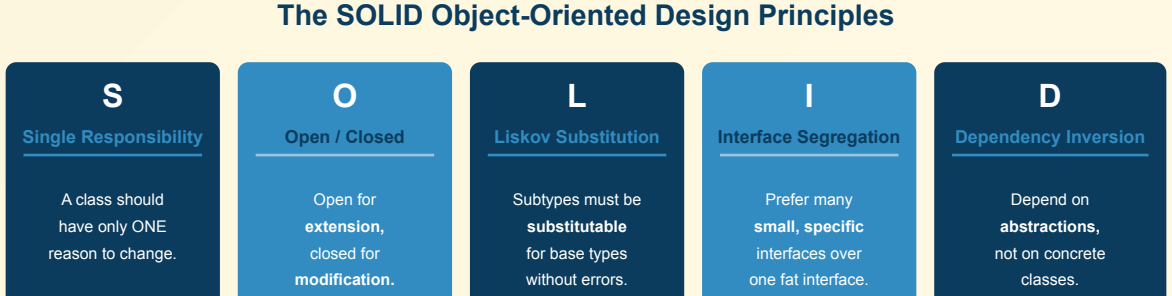


5.6 SOLID Object-Oriented Principles

> *"Clean code always looks like it was written by someone who cares."*
> *— Robert C. Martin (Uncle Bob)*

Overview: The SOLID Principles

- **S – Single Responsibility Principle (SRP):**
  - A class should have one, and only one, reason to change.
- **O – Open / Closed Principle (OCP):**
  - Software entities should be open for extension, but closed for modification.
- **L – Liskov Substitution Principle (LSP):**
  - Subtypes must be substitutable for their base types without altering program correctness.
- **I – Interface Segregation Principle (ISP):**
  - Clients should not be forced to depend on interface methods they do not use.
- **D – Dependency Inversion Principle (DIP):**
  - High-level modules should not depend on low-level modules; both should depend on abstractions.



S – Single Responsibility Principle (SRP)

- **Core Principle:**

> *A class should have one, and only one, reason to change.*
>
> ”

- **Why SRP is Vital:**
  - When classes combine multiple responsibilities (e.g., database storage + PDF reporting + email dispatch), a change in email formatting risks introducing bugs into database persistence logic!

```
// ❌ Violates SRP: 3 distinct reasons to change (DB, PDF formatting, Email delivery)
class UserReportManager {
    public void fetchUserData() { /* DB SQL */ }
    public void generatePdfReport() { /* PDF rendering */ }
    public void sendEmailReport() { /* SMTP protocol */ }
}

// ✅ Follows SRP: 3 separate, single-purpose classes
class UserRepository { public User fetchUserData() { ... } }
class PdfReportFormatter { public byte[] generatePdf(User u) { ... } }
class EmailService { public void sendEmail(String to, byte[] data) { ... } }
```

O – Open / Closed Principle (OCP)

- **Core Principle:**

> *Software entities should be open for extension, but closed for modification.* ”

- **The Engineering Mechanism:**
  - Use **polymorphism and interfaces** (e.g., Strategy Pattern) rather than fragile `if-else` or `switch` statements checking type codes.

```
// ❌ Violates OCP: Adding a new discount tier REQUIRES modifying existing tested code!
class DiscountCalculator {
    public double calculate(String tier, double price) {
        if (tier.equals("VIP")) return price * 0.8;
        else if (tier.equals("STUDENT")) return price * 0.9;
        return price; // Must edit this method every time a new promotion is invented!
    }
}

// ✅ Follows OCP: Add new tiers by creating NEW classes implementing the interface!
interface DiscountStrategy { double apply(double price); }
class VipDiscount implements DiscountStrategy { public double apply(double p) { return p * 0.8; } }
class StudentDiscount implements DiscountStrategy { public double apply(double p) { return p * 0.9; } }
```

L – Liskov Substitution Principle (LSP)

- **Core Principle:**

> *Subtypes must be substitutable for their base types without altering program correctness.*
>
> ”

- **The Classic Anti-Pattern: Square extending Rectangle:**
  - In geometry, a square is a rectangle. But in object-oriented code, setting the width of a Square must also change its height, breaking client code expectations!

```
// ❌ Violates LSP: Square breaks the invariant assumptions of Rectangle!
class Rectangle {
    protected int width, height;
    public void setWidth(int w) { this.width = w; }
    public void setHeight(int h) { this.height = h; }
    public int getArea() { return width * height; }
}
class Square extends Rectangle {
    @Override public void setWidth(int w) { this.width = w; this.height = w; }
    @Override public void setHeight(int h) { this.width = h; this.height = h; }
}
```

I – Interface Segregation Principle (ISP)

- **Core Principle:**

> *Clients should not be forced to depend on interface methods they do not use.*

- **The "Fat Interface" Problem:**
  - Avoid creating massive interfaces with dozens of methods. Instead, favor small, focused, role-specific interfaces.

```
// ❌ Violates ISP: "Fat Interface" forces SimplePrinter to implement methods it cannot support!
interface MultiFunctionDevice {
    void print();
    void scan();
    void fax();
}
class SimplePrinter implements MultiFunctionDevice {
    public void print() { /* OK */ }
    public void scan() { throw new UnsupportedOperationException(); } // Anti-pattern!
    public void fax() { throw new UnsupportedOperationException(); }
}

// ✅ Follows ISP: Segregated, role-specific interfaces
```

D – Dependency Inversion Principle (DIP)

- **Core Principle:**

> *High-level modules should not depend on low-level modules. Both should depend on abstractions.*
>
> ”

- **The Engineering Solution: Dependency Injection (DI):**
  - High-level business services must never instantiate concrete database classes directly with `new MySQLDatabase()`. Instead, inject an abstract `Database` interface!

```
// ❌ Violates DIP: High-level business logic is hardcoded to concrete MySQL implementation!
class OrderService {
    private MySQLDatabase db = new MySQLDatabase(); // Tight coupling! Cannot mock for tests!
    public void completeOrder(Order o) { db.insert(o); }
}

// ✅ Follows DIP: Both depend on abstract interface; injected via constructor
interface Database { void insert(Order o); }

class OrderService {
    private final Database db; // Depends strictly on abstraction!
```

[MICRO_LEVEL] // CODE DIAGNOSTICS

SOLID: The Good vs. Bad Matrix

Principle	❌ The Violation	✅ The Fix	Core Benefit
SRP	Changes to PDF layout and Email server break the same class.	Separate PDF and Email into distinct classes.	Isolates requirement changes.
OCP	Using massive if-else or switch blocks on object types.	Use Polymorphism and interfaces (Strategy Pattern).	Add features without editing tested code.
LSP	Subclasses throwing unexpected exceptions breaking base assumptions.	Derived classes strictly fulfill base invariants.	Prevents polymorphic runtime crashes.
ISP	Dummy method implementations throwing `UnsupportedOperationException`.	Segregate into multiple small, focused interfaces.	Prevents forced dependency on unused methods.
DIP	High-level business logic instantiates concrete database engines.	Use Dependency Injection (DI) and depend on interfaces.	Allows swapping services without rewriting logic.

NotebookLM

Concept Check Question 4

Which SOLID design principle states that you should be able to introduce new system capabilities by adding new classes without editing existing, tested source code?

- **A.** Single Responsibility Principle (SRP)
- **B.** Open / Closed Principle (OCP)
- **C.** Interface Segregation Principle (ISP)
- **D.** Liskov Substitution Principle (LSP)

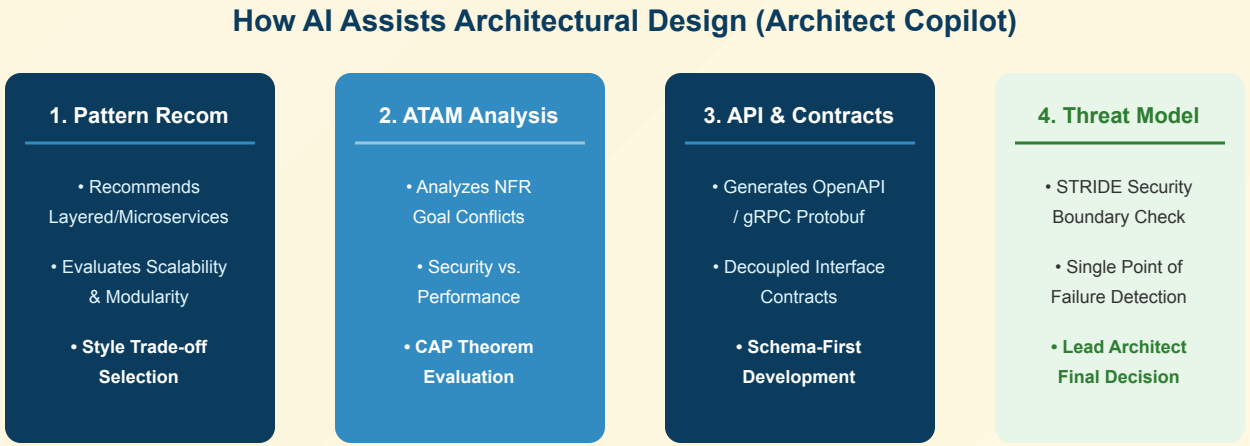


5.7 AI in Architecture & Software Design

> *"AI can recommend design patterns and refactor code smells, but human architects must steer the system's destiny."*

AI in Architecture & Design: The Copilot Paradigm

- **The Analytical Copilot Leap:**
  - LLMs analyze complex system specifications to recommend suitable architectural patterns (e.g., Microservices vs. Event-Driven).
  - Rapidly generates interface contracts (OpenAPI / Swagger specs, gRPC `.proto` schemas, and database DDL).



- **Core Architectural Value:** Accelerates trade-off analysis and generates clean boilerplate interfaces between sub-systems.

4 Core AI Applications in Software Design

- **1. Automated SOLID Refactoring:**
  - Scans legacy classes to detect 'God Classes' (SRP violations) and suggests Strategy or Factory patterns to achieve OCP.
- **2. Code Smell & Coupling Detection:**
  - Identifies tight coupling, circular dependencies, duplicated logic, and shotgun surgery hotspots across repositories.
- **3. Design Pattern Recommendation:**
  - Recommends GoF design patterns (Observer, Decorator, Adapter, Builder) for recurring architectural friction points.
- **4. Interface & Mock Generation for TDD:**
  - Auto-generates decoupled interfaces and mock objects, accelerating Test-Driven Development pipelines.

Human-in-the-Loop: Architectural Blind Spots

- **Risks of Unchecked AI in Architecture & Design:**
  - **Premature Distributed Complexity:** Recommending complex microservices and event brokers for simple CRUD applications that would thrive as clean monoliths.
  - **Context Blindness:** AI lacks awareness of team engineering skillsets, cloud infrastructure budgets, and corporate regulatory hurdles.
  - **Superficial Refactoring:** Renaming variables without resolving underlying algorithmic or architectural coupling.
- **The Golden Architectural Rule:**

> ***AI Proposes Trade-offs; Lead Architects Make Decisions!***
> *Human engineers must evaluate organizational context and maintain ultimate responsibility for system viability.*

5.8 Conceptual Recap & References

> *"Architecture is the decisions that you wish you could get right the first time."* „

Conceptual Recap: Fill-in-the-blank Quiz

Test your understanding of the core concepts in this chapter:

1. ___ architecture structures a system into stacked layers where each layer provides services to the layer directly above.
2. In the ___ pattern, multiple independent sub-systems interact entirely through a shared central database.
3. Kruchten's ___ View Model presents Logical, Process, Development, and Physical views joined by Scenarios.
4. ___ measures the internal focus of a class, while ___ measures the degree of dependency between classes.
5. The ___ Principle states that classes should be open for extension but closed for modification.
6. The ___ Principle requires that high-level business logic must depend on abstractions rather than concrete classes.

References & Further Reading

- **Foundational Textbooks & Classics:**
  - Sommerville, I. (2016). *Software Engineering* (10th ed.). Chapter 6 (Architecture) & Chapter 7 (Design). Pearson.
  - Kruchten, P. (1995). "Architectural Blueprints — The '4+1' View Model of Software Architecture." *IEEE Software*, 12(6), 42-50.
  - Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
  - Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- **Modern Distributed Systems & Architecture:**
  - Fowler, M. (2014). *Microservices: A definition of this new architectural term*. martinfowler.com
  - Richards, M., & Ford, N. (2020). *Fundamentals of Software Architecture*. O'Reilly Media.